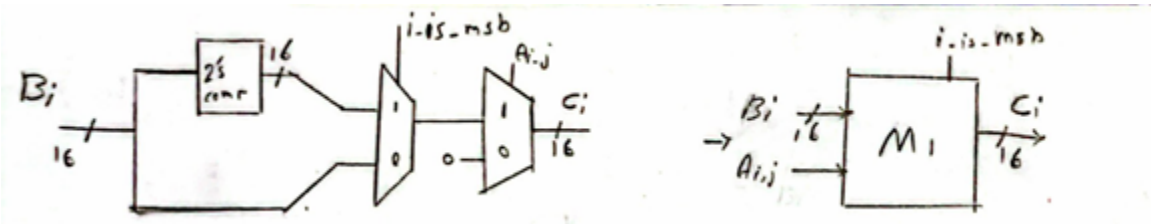
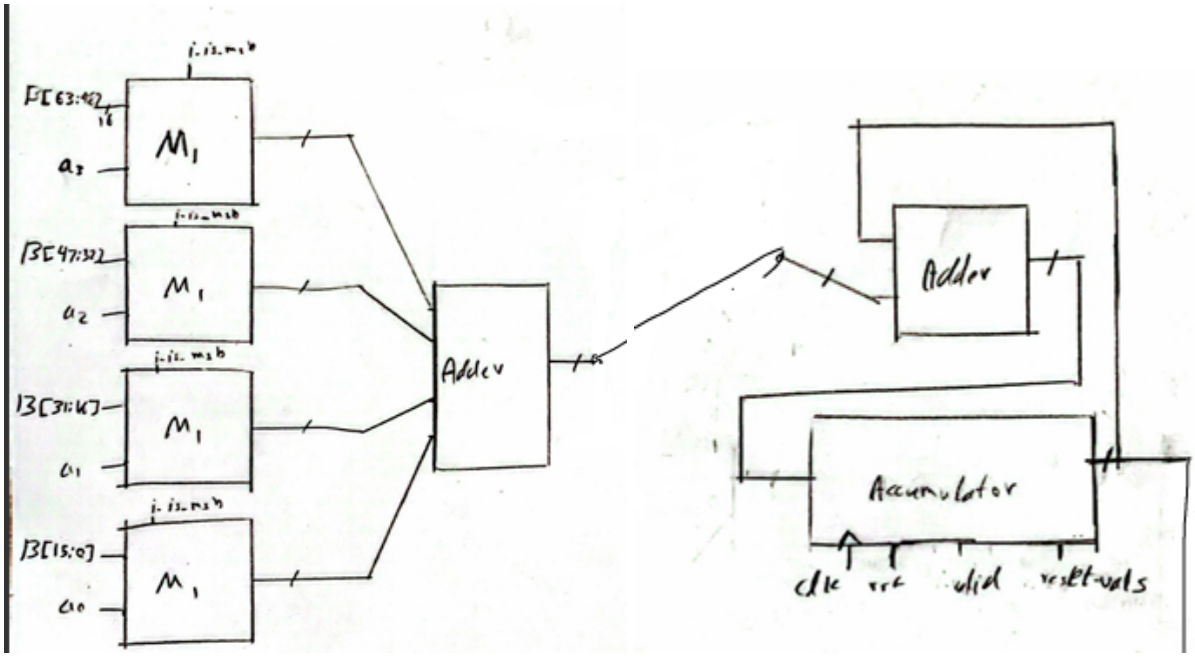
نام و نام خانوادگی: دنیز مصطفی زادگان
شماره دانشجویی: 810102603
نام و نام خانوادگی: سید حمید محمودی
شماره دانشجویی: 810102549

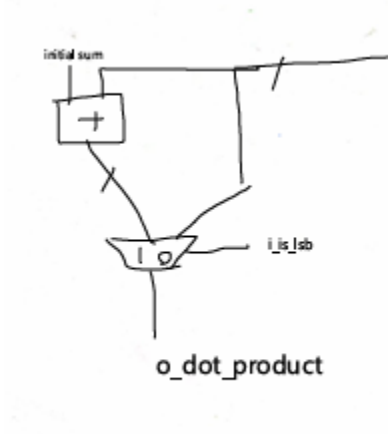
CAD_CA2_REPORT

In SPE we need to use add and shift multiplier but for more than one numbers, so if we are on the first bit of a we should take -B else we should take B itself, and then use that numbers and if we have a as 1 we use it, otherwise we take 0, after that we sum up all these numbers, in example is 4, and sum it up with the output of accumulator and give it to it as an input, it makes it like accumulator has been shifted by one and summed up by the new input:

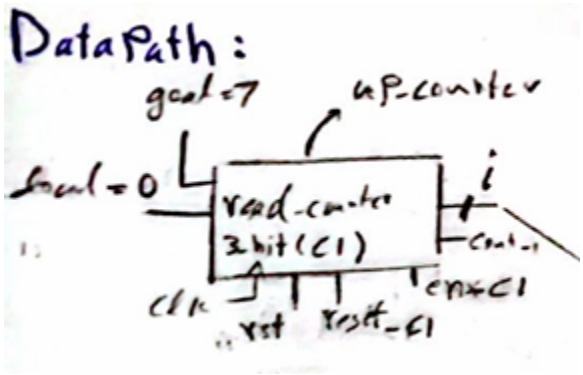




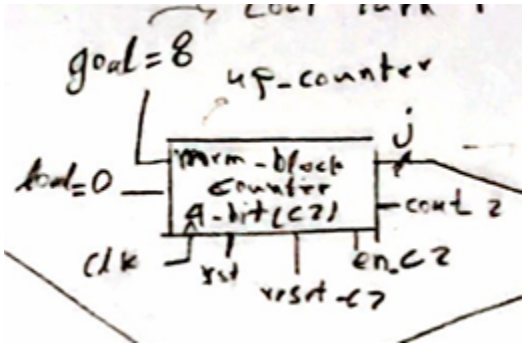
After this if are on lsb the output should be accumulator_out put + initial sum, otherwise the accumulator output itself:



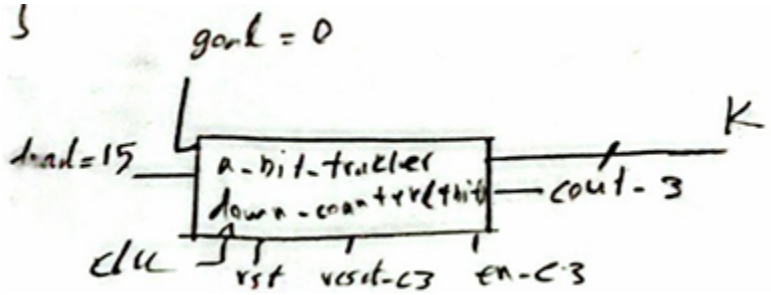
Before this we used to also register initial sum at the last bit but a problem arose when we tried to run two SPE's simultaneously, the output of the first SPE which was the initial sum of the second one was one clock late to generate the correct initial sum, sum had to make it the way we had, and also needed to turn on lsb in another state too in order to write the SPE1+SPE2 outs. But first let's take a look at the matrix multiplication unit:
We have 3 main counters
One moves in a block to read from the memory(up counter)



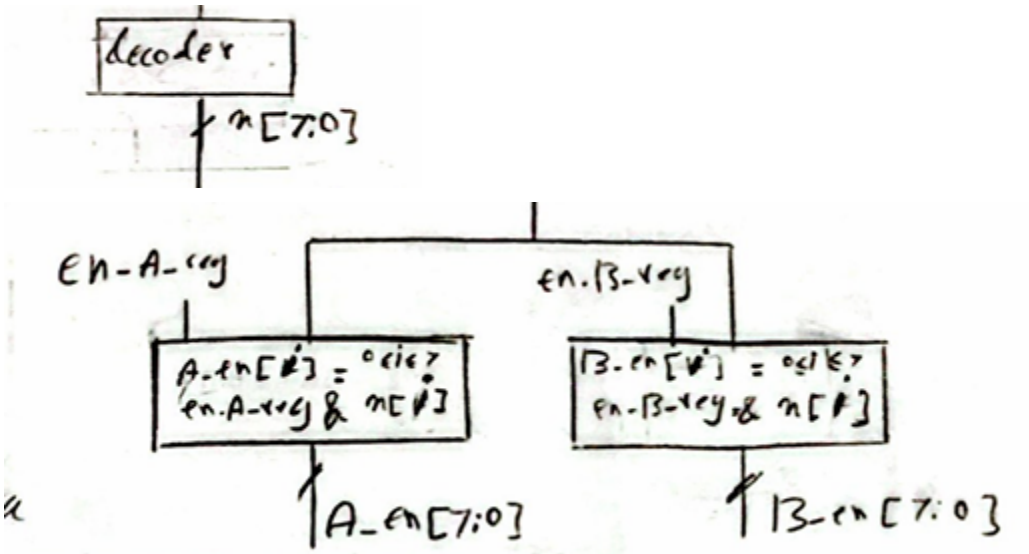
One changes the block to allocate the block which needs to be read(up counter)

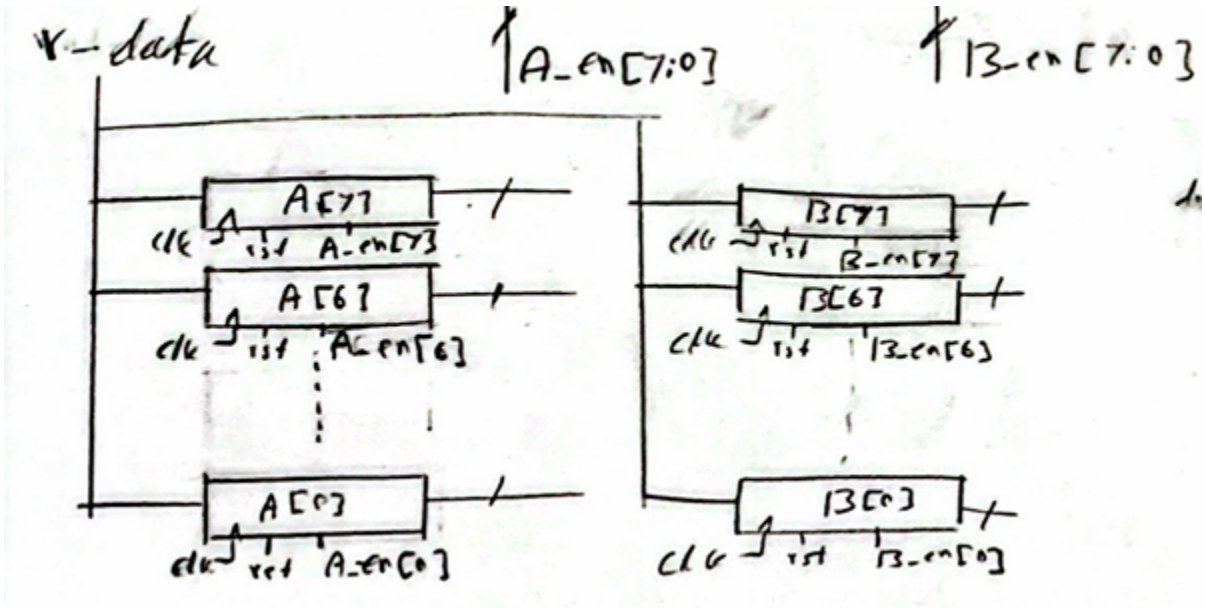


And last one moves on A registers bits to make the a_vec input for SPE's(down counter)

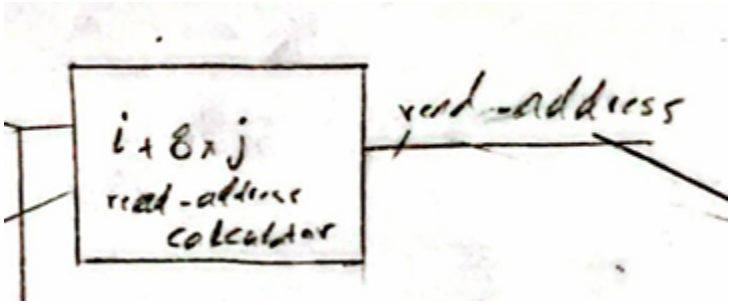


First we read A once, and then each row of B each main cycle to calculate that row of C
But in order to load into registers we need to know which register we are on and we use a decoder and a en to set enable for loading registers which is anded with the decoder output for A and B.

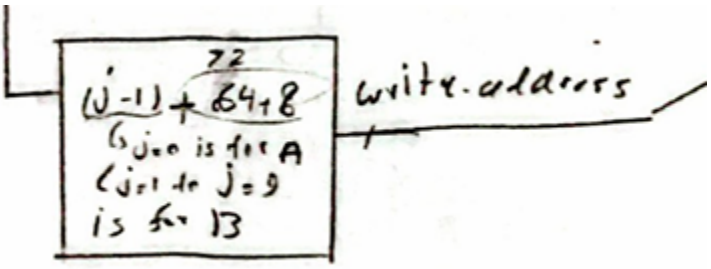




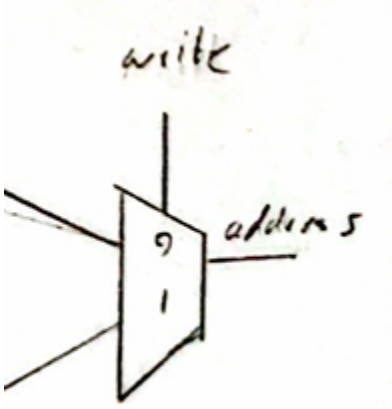
And the read address simply comes from block_itt + block_num*block_size(Or rows of matrix)



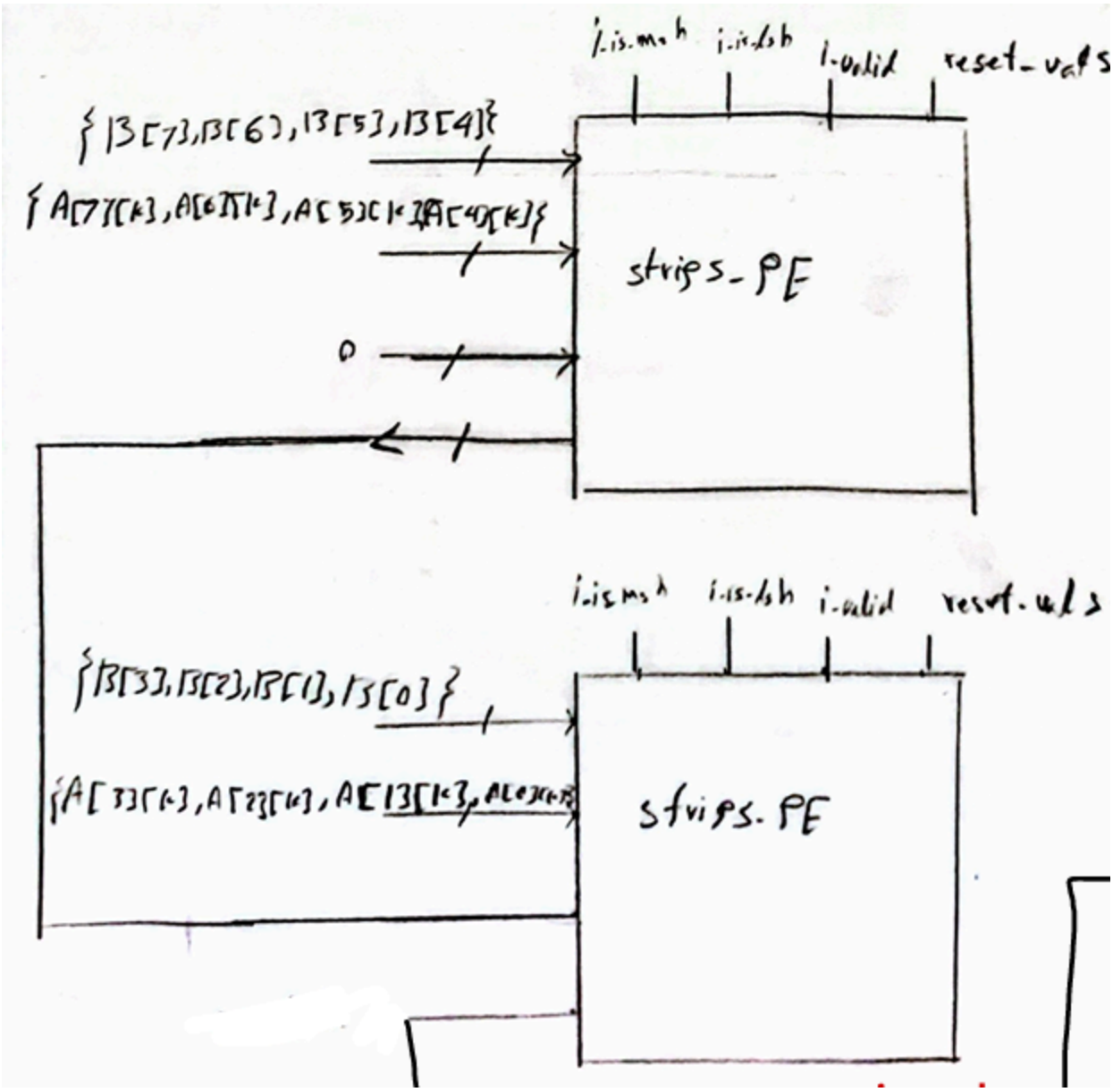
And the write address come from (block_num-1)+72 (num of rows * (num of rows + 1))



And using write signal we choose what address to send to memory where when write is 1 we send write_address and when its 0 we send read_address



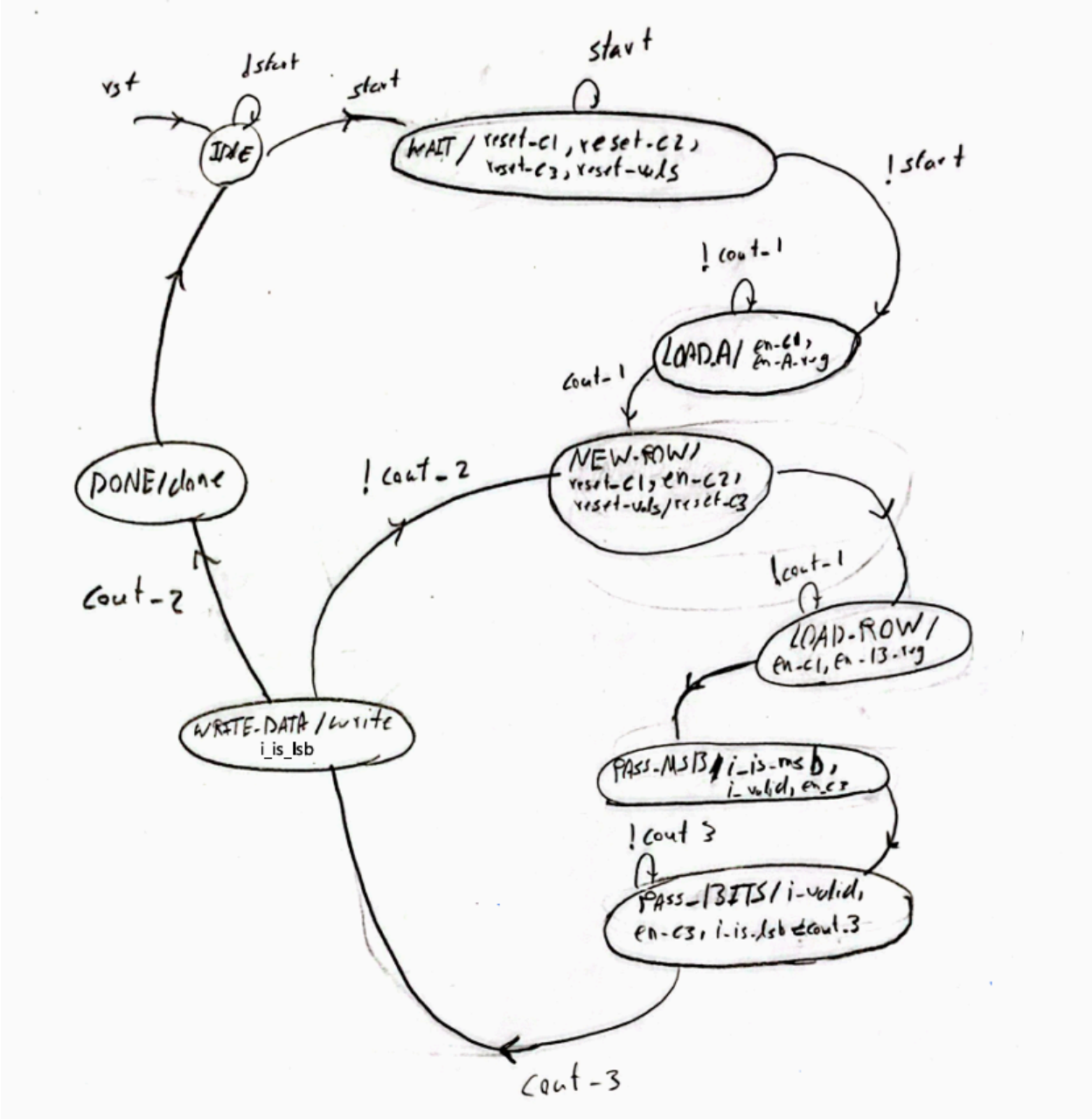
And finally we send have of B and A's to the first SPE and the rest half to the second one:



And the output of the second SPE in write_data

Also we activate reset_vals each cycle to clear the values of the last cycle from the accumulator

Just like the we reach this FSM:



A single signal had been added and its the extra i_is_lsb in order to get the correct output when writing the data into memory.

First example which gets us the correct values and saves them in the memory too can be seen down here:

